

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: *Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO (BL5, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct:

I Kanmani L/192511414 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

CONSTRAINT-BASED PROBLEM SOLVING

1. Intrusion Detection Deployment under Real-Time Constraints

A financial organization plans to deploy an Intrusion Detection System (IDS) with the following constraints:

- Must analyze traffic in real-time with latency ≤ 5 ms per packet
- Detection accuracy $\geq 96\%$
- False positive rate $\leq 1.5\%$
- Network traffic load: 150,000 packets per second
- Limited CPU resources (shared with other services)

Task:

Design an optimized IDS configuration (signature-based, anomaly-based, or hybrid). Analyze trade-offs between detection accuracy and system performance, and justify how your solution meets all constraints.

2. Worm Containment Strategy under Network Availability Constraints

An enterprise network faces rapid worm propagation with the following constraints:

- Network downtime must be $\leq 1\%$
- Detection and containment must occur within 2 minutes
- Systems are distributed across multiple locations
- Limited centralized control

Task:

Design a containment strategy using firewalls, IDS/IPS, and segmentation. Evaluate trade-offs between rapid isolation and business continuity, and justify how your approach minimizes spread while maintaining network availability.

3. Firewall Rule Optimization under Performance Constraints

A company firewall currently uses 10,000+ rules, causing performance degradation. Constraints include:

- Throughput requirement: ≥ 800 Mbps
- Rule processing time must be minimized
- Must maintain strict access control policies
- No additional hardware allowed

Task:

Analyze inefficiencies in rule processing. Propose optimization techniques (rule aggregation, prioritization, redundancy elimination). Justify how your optimized rule set improves performance while maintaining security.

4. IPS Deployment under Resource and Accuracy Constraints

An organization wants to deploy an Intrusion Prevention System (IPS) with the following constraints:

- Must block $\geq 90\%$ of known attacks
- False positive rate $\leq 3\%$
- Available memory for signatures: 1 GB
- Network speed: 500 Mbps

Task:

Design an IPS configuration strategy (signature selection, traffic filtering, adaptive updates). Evaluate trade-offs between resource usage and detection capability, and justify how your solution satisfies all constraints.

5. Malware Detection using Behavioral Analysis under System Constraints

A company plans to implement behavioral-based malware detection with the following constraints:

- Endpoint CPU usage must remain $\leq 15\%$
- Detection must occur within 10 seconds of execution
- Must detect unknown (zero-day) threats
- Limited storage for logs and analysis

Task:

Design a lightweight behavioral detection system. Analyze trade-offs between detection speed, accuracy, and resource consumption, and justify how your approach effectively identifies malware within the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Intrusion Detection Deployment under Real-Time Constraints

Answer: Proposed solution: Hybrid IDS

A hybrid IDS combining signature-based and lightweight anomaly-based detection is most suitable. Signature detection handles known attacks quickly, while anomaly detection identifies unusual or zero-day behavior.

Configuration

- Use signature-based detection as the first layer because it has low processing overhead.
- Maintain a small, high-confidence signature database containing the most important attack patterns.
- Apply anomaly detection only to suspicious traffic rather than every packet.
- Use packet filtering to remove trusted or irrelevant traffic before deep inspection.
- Use multiple processing threads/cores where available, while reserving CPU capacity for other services.
- Update signatures periodically rather than continuously during peak traffic.

Processing flow

Incoming Traffic → Packet Filtering → Signature Detection → Suspicious Packets → Lightweight Anomaly Analysis → Alert/Block

Trade-offs

Approach	Accuracy	Speed	CPU Usage	Zero-day Detection
Signature-based	High for known attacks	Very high	Low	Poor
Anomaly-based	High potential	Lower	High	Good
Hybrid	High	High	Moderate	Good

Using anomaly detection on all 150,000 packets/sec could exceed the CPU and 5-ms latency constraints. Therefore, the anomaly module is activated only after signature/filtering identifies suspicious traffic.

Meeting the constraints

- Latency ≤ 5 ms: Fast filtering and signature matching are performed first.
- Accuracy $\geq 96\%$: Combining known-attack signatures with anomaly detection improves coverage.
- False positives $\leq 1.5\%$: Use high-confidence signatures and calibrated anomaly thresholds.
- 150,000 packets/sec: Lightweight first-stage processing prevents expensive analysis of every packet.
- Limited CPU: Deep inspection is restricted to suspicious traffic.

A hybrid IDS with signature-first filtering and selective anomaly analysis provides the best balance between detection accuracy and real-time performance. It can maintain high detection capability without continuously consuming excessive CPU resources.

2. Worm Containment Strategy under Network Availability Constraints

Answer:

Proposed solution: Distributed segmentation with IDS/IPS and firewalls

The organization should use a distributed containment architecture rather than depending on one centralized controller.

Strategy

1. Deploy IDS/IPS sensors at major network entry points and between important network segments.
2. Divide the network into segments such as:
 - User network
 - Server network
 - Database network
 - Guest network
 - Critical systems
3. Configure firewalls between these segments.
4. When the IDS detects worm-like behavior, such as rapid scanning or abnormal connection rates, the IPS automatically blocks the malicious source.
5. Only the affected segment is isolated initially.
6. Other segments continue operating normally.
7. Local security devices can make containment decisions even if centralized management is unavailable.

Flow

Worm Activity → IDS Detection → IPS Verification → Block Malicious Source → Isolate Affected Segment → Monitor Other Segments → Restore Connectivity

Trade-offs

Rapid isolation:

Advantages:

- Stops worm propagation quickly.
- Reduces the number of infected systems.
- Helps meet the 2-minute containment requirement.

Disadvantage:

- Excessive isolation can interrupt legitimate business operations.

Gradual isolation:

Advantages:

- Better network availability.
- Less disruption.

Disadvantage:

- Allows the worm more time to spread.

Meeting the constraints

- Downtime $\leq 1\%$: Isolate only the infected VLAN/subnet instead of shutting down the entire network.
- Containment ≤ 2 minutes: Automated IDS/IPS rules immediately block suspicious traffic.
- Multiple locations: Each location has local firewall and IDS/IPS controls.
- Limited centralized control: Local security devices can operate independently.

A distributed, segmented network with automated IDS/IPS containment provides rapid worm control while preserving business continuity. Selective isolation is better than complete network shutdown because it minimizes both malware spread and downtime.

3. Firewall Rule Optimization under Performance Constraints

Answer: A firewall with more than 10,000 rules may experience performance degradation because packets must be compared against many rules before a match is found.

Main inefficiencies:

1. Redundant rules perform the same function.
2. Duplicate rules unnecessarily increase processing.
3. Broad rules may appear before specific rules and cause incorrect or inefficient matching.
4. Obsolete rules consume processing time.
5. Poorly organized rules increase average packet-processing time.
6. Large groups of similar rules can be combined.

Proposed optimization

1. Rule aggregation

Combine rules having the same action and similar source/destination addresses.

For example:

192.168.1.10 → Server A → Allow
192.168.1.11 → Server A → Allow
192.168.1.12 → Server A → Allow

can potentially be represented using an appropriate subnet/network rule.

2. Remove redundant rules

Identify rules that are completely covered by earlier rules and remove them.

3. Remove obsolete rules

Delete rules related to:

- Decommissioned systems
- Expired temporary access
- Old applications
- Unused ports

4. Rule prioritization

Place frequently matched rules near the top, while keeping security-specific exceptions before broader rules where required by policy.

5. Group similar rules

Organize rules according to:

- Source
- Destination
- Protocol
- Application
- Security zone

Optimized flow

Incoming Packet → Common/High-Frequency Rules → Specific Security Rules → Aggregated Rules → Default Deny

Security consideration

Optimization must not simply remove rules to improve speed. Every deleted or merged rule should be checked against the organization's access-control policy.

Expected result:

Before optimization	After optimization
10,000+ rules	Smaller optimized rule set
More comparisons	Fewer comparisons
Higher processing time	Lower processing time
Lower throughput	Improved throughput
Difficult maintenance	Easier management

The target is to reduce the average number of rules examined per packet sufficiently to maintain ≥ 800 Mbps throughput without weakening the access-control policy.

Conclusion

Rule aggregation, redundancy elimination, obsolete-rule removal, and intelligent prioritization reduce firewall processing overhead. This improves throughput and maintains strict security without requiring additional hardware.

4. IPS Deployment under Resource and Accuracy Constraints

Answer: Proposed solution: Optimized signature-based IPS with selective inspection

Since the organization has only 1 GB memory for signatures, storing every available signature is inefficient. The IPS should use a carefully selected signature database.

Configuration

1. Signature selection

Prioritize:

- High-severity attacks
- Frequently observed attacks
- Exploits targeting the organization's operating systems
- Web attacks
- Malware communication
- Known vulnerabilities

Low-confidence and irrelevant signatures can be disabled.

2. Traffic filtering

Before signature inspection:

Traffic → Protocol/Port Filtering → Signature Matching → Threat Verification → Block/Allow

Unnecessary traffic can be filtered before expensive inspection.

3. Signature optimization

Use:

- Compact signatures
- Efficient pattern matching
- Duplicate signature removal
- Grouped signatures
- Application-specific signatures

4. Adaptive updates

Update signatures based on:

- Newly discovered attacks
- Current threat intelligence
- Organization-specific attack patterns

Old or irrelevant signatures can be removed or disabled.

Trade-offs

Strategy	Detection	Memory	Performance
All signatures	Very high	Very high	Lower
Selected signatures	High	Low	High
Very few signatures	Lower	Very low	Very high

A carefully selected signature set provides better performance while retaining protection against the most important threats.

Meeting constraints

- $\geq 90\%$ known attacks: Prioritize signatures covering common and high-risk attacks.
- $\leq 3\%$ false positives: Use high-confidence signatures and verification before blocking.
- 1 GB memory: Store only relevant, optimized signatures.
- 500 Mbps: Perform lightweight traffic filtering before deep signature inspection.

signature-based IPS is appropriate because the attack requirement concerns known attacks. Filtering, signature prioritization, and adaptive updates allow the IPS to remain within the 1-GB memory limit while maintaining the required detection and network performance

5. Malware Detection Using Behavioral Analysis under System Constraints

Answer: Proposed solution: Lightweight endpoint behavioral detector

A lightweight behavioral system can detect malware by monitoring behavior rather than relying only on known malware signatures. This makes it useful against zero-day threats.

Monitored behaviors

The system can monitor:

- Unusual process creation
- Abnormal file modifications
- Registry/configuration changes
- Suspicious network connections
- Privilege escalation attempts
- Rapid file encryption
- Unusual child-process creation
- Access to sensitive system resources

Detection architecture

Process Execution → Lightweight Behavior Monitoring → Feature Extraction → Risk Score → Threshold Check → Alert/Terminate

Instead of continuously storing complete logs, the system maintains only compact behavioral events and risk scores.

Resource optimization

$\text{CPU} \leq 15\%$

- Monitor only security-sensitive events.
- Use event-driven monitoring rather than continuous scanning.
- Perform lightweight feature extraction.
- Run deeper analysis only when the risk score crosses a threshold.

Detection within 10 seconds

Use short observation windows. For example, suspicious process behavior can be evaluated continuously and an action can be taken as soon as enough evidence is collected.

Limited storage

Store:

- Process ID
- Event type

- Timestamp
- Risk score
- Important network/file indicators

Avoid storing complete copies of every event.

Zero-day detection

Signature-based systems may fail when malware has never been seen before. Behavioral analysis instead identifies malicious patterns, such as:

Unknown application → creates many files → modifies system settings → contacts suspicious external host

Even without knowing the malware's name or signature, this combination can generate a high risk score.

Trade-offs

Method	Detection Speed	Resource Usage	Zero-day Detection
Signature-based	Very fast	Low	Poor
Full behavioral analysis	Moderate	High	Excellent
Lightweight behavioral analysis	Fast	Low–moderate	Good

Meeting constraints

- CPU $\leq 15\%$: Event-driven and selective monitoring.
- Detection ≤ 10 seconds: Continuous short-window behavioral scoring.
- Zero-day detection: Detects suspicious behavior instead of depending only on signatures.
- Limited storage: Store compressed/high-value events rather than full activity logs.

A lightweight, event-driven behavioral malware detector provides a good balance between detection capability and resource consumption. Selective monitoring and risk-based escalation allow zero-day threats to be detected quickly while keeping endpoint CPU usage and storage within the specified limits.